STUDENT MANAGEMENT SYSTEM

USING PYTHON

Mini Project Report:

Submitted by

Budala Roja Rani

B.Tech – Computer Science and Engineering

Department of Computer Science and Engineering

RISE Krishna Sai Prakasam Group of Institutions

Academic Year:2025-2026

CERTIFICATE

This is to certify that Mr./Ms. BUDALA ROJA RANI, a student of B.Tech in Computer Science and Engineering, has successfully completed the mini project titled "Student Management System Using Python" during the academic year 2025–2026.TheDepartment of Computer Science and Engineering.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide and faculty members for their valuable guidance and support throughout the completion of this project.

I also thank my college for providing the necessary facilities and resources required for this project. Finally, I would like to thank my family and friends for their encouragement and support.

ABSTRACT

The Student Management System is a Python-based application developed to simplify the management of student records in an educational institution. The system allows users to add, search, update, delete, and view student details efficiently. It also provides a login system for secure access and uses file handling to store student information permanently.

The system reduces manual paperwork, improves data accuracy, and helps maintain student records in an organized and efficient manner. This project is developed using Object-Oriented Programming (OOP) concepts and provides a simple menu-driven interface, making it easy for users to manage student information effectively.

INTRODUCTION

Managing student records manually can be difficult and time-consuming. Maintaining information such as student details, attendance, marks, and academic records using paper registers often leads to errors, data duplication, and loss of information.

The Student Management System is developed using Python to simplify these tasks. The system provides a user-friendly, menu-driven interface that allows users to add, search, update, delete, and view student records quickly and efficiently.

The application stores student information in a text file, ensuring that records are saved permanently even after the program is closed. This system reduces manual work, improves accuracy, saves time, and helps educational institutions manage student records in an organized and efficient manner.

INTRODUCTION

Managing student records manually can be difficult and time-consuming. Maintaining information such as student details, attendance, marks, and academic records using paper registers often leads to errors, data duplication, and loss of information.

The Student Management System is developed using Python to simplify these tasks. The system provides a user-friendly, menu-driven interface that allows users to add, search, update, delete, and view student records quickly and efficiently.

The application stores student information in a text file, ensuring that records are saved permanently even after the program is closed. This system reduces manual work, improves accuracy, saves time, and helps educational institutions manage student records in an organized and efficient manner.

PROPOSED SOLUTION

The proposed Student Management System is developed using Python to automate the process of managing student records in an efficient and organized manner. The system replaces the traditional manual method of maintaining student information, reducing paperwork and minimizing errors.

The application provides a simple menu-driven interface with secure login authentication. It allows users to add new student records, view all student details, search for a student, update existing records, delete student records, and display the total number of students. The system also validates user input to ensure accurate data entry.

Student information is stored permanently using file handling, allowing records to be saved and retrieved whenever the application is reopened. This ensures data persistence and easy access to student information.

Overall, the proposed system improves accuracy, saves time, reduces manual effort, and provides an effective solution for managing student records in educational institutions.

TECHNOLOGIES USED

- Python
- Object-Oriented Programming (OOP)
- Functions
- Lists and Dictionaries
- File Handling
- Conditional Statements
- Loops
- Text File Storage

SYSTEM DESIGN

User → Login System → Student Management System → Student Operations → Student Records

The user logs into the system using a username and password. After successful authentication, the user can perform various operations such as adding, viewing, searching, updating, and deleting student records. All student information is stored permanently using file handling, allowing records to be accessed whenever the application is reopened.

MODULES Login Module

This module verifies the username and password before allowing access to the system, ensuring that only authorized users can manage student records.

Add Student Module

This module is used to add new student records with details such as Student ID, Student Name, Age, Gender, Course, Department, and Contact Number.

View Students Module

This module displays all student records stored in the system in an organized format.

Search Student Module

This module helps users search for a student using the unique Student ID and displays the corresponding student details.

Update Student Module

This module allows users to modify existing student information, such as name, course, department, contact number, or other details.

Delete Student Module

This module is used to remove a student record from the system using the Student ID.

Attendance Module

This module is used to record and manage student attendance and view attendance details.

Marks Management Module

This module stores and updates students' marks or grades for different subjects.

Student Count Module

This module displays the total number of student records available in the system.

File Handling Module

This module stores student records permanently in a file and retrieves the data whenever the application is reopened.

Logout Module

This module allows users to exit the system securely after completing their operations.

ALGORITHM

1. Start the program.
2. Display the login screen.
3. Verify the username and password.
4. Load student records from the file.
5. Display the main menu options.
6. Read the user's choice.
7. Perform the selected operation (Add, View, Search, Update, Delete, or Count Student Records).
8. Save the updated student records to the file.
9. Return to the main menu until the user chooses to exit.
10. Exit the program.

Python Code

```
#==============================
#STUDENT MANAGEMENT SYSTEM
#==============================
import re


class StudentManagementSystem:

    FILE_NAME = "students.txt"

    def create_login(self):
        print("\n========== CREATE
ACCOUNT ==========")

        while True:
            username = input("Create
Username: ")
            password = input("Create
Password: ")

            # Username: letters, numbers and
special characters allowed
```

```
        # Password: capital, small,
numbers and special characters allowed

        if len(username) < 4:
            print("Username must contain
at least 4 characters.")
            continue

        if len(password) < 8:
            print("Password must contain
at least 8 characters.")
            continue

        if not re.search(r"[A-Z]",
password):
            print("Password must contain
at least one Capital letter.")
            continue

        if not re.search(r"[a-z]",
password):
            print("Password must contain
at least one Small letter.")
```

```
            continue

        if not re.search(r"[0-9]",
password):
            print("Password must contain
at least one Number.")
            continue

        if not re.search(r"[^A-Za-z0-9]",
password):
            print("Password must contain
at least one Special character.")
            continue

        print("\nAccount created
successfully!")
        return username, password

    def login(self, username, password):
        print("\n========== LOGIN
==========")
```

```
        login_username = input("Enter
Username: ")
        login_password = input("Enter
Password: ")

        if login_username == username and
login_password == password:
            print("\nLogin successful!")
            return True
        else:
            print("\nInvalid Username or
Password!")
            return False

    def add_student(self):
        print("\n========== ADD STUDENT
==========")

        student_id = input("Enter Student
ID: ")
        name = input("Enter Student Name:
")
        age = input("Enter Age: ")
```

```
        course = input("Enter Course: ")
        marks = input("Enter Marks: ")

        with open(self.FILE_NAME, "a") as
file:
            file.write(

f"{student_id},{name},{age},{course},{m
arks}\n"
            )

        print("Student added successfully!")

    def view_students(self):
        print("\n========== STUDENT
DETAILS ==========")

        try:
            with open(self.FILE_NAME, "r") as
file:
                students = file.readlines()

            if not students:
```

```
            print("No student records
found.")
            return

        for student in students:
            data = student.strip().split(",")

            print("----------------------------")
            print("Student ID :", data[0])
            print("Name       :", data[1])
            print("Age        :", data[2])
            print("Course     :", data[3])
            print("Marks      :", data[4])

    except FileNotFoundError:
        print("No student records
found.")

  def search_student(self):
    print("\n========== SEARCH
STUDENT ==========")
```

```
        search_id = input("Enter Student ID:
")

        try:
            with open(self.FILE_NAME, "r") as
file:
                for student in file:
                    data =
student.strip().split(",")

                    if data[0] == search_id:
                        print("\nStudent Found!")
                        print("Student ID :",
data[0])
                        print("Name      :", data[1])
                        print("Age       :", data[2])
                        print("Course    :", data[3])
                        print("Marks     :", data[4])
                        return

            print("Student not found.")

        except FileNotFoundError:
```

```
            print("No student records
found.")

    def update_student(self):
        print("\n========== UPDATE
STUDENT ==========")

        update_id = input("Enter Student
ID: ")

        try:
            with open(self.FILE_NAME, "r") as
file:
                students = file.readlines()

            found = False

            with open(self.FILE_NAME, "w")
as file:
                for student in students:
                    data =
student.strip().split(",")
```

```
                if data[0] == update_id:
                    name = input("Enter New
Name: ")
                    age = input("Enter New
Age: ")
                    course = input("Enter New
Course: ")
                    marks = input("Enter New
Marks: ")

                    file.write(

f"{update_id},{name},{age},{course},{ma
rks}\n"
                    )

                    found = True
                else:
                    file.write(student)

            if found:
                print("Student updated
successfully!")
```

```
            else:
                print("Student not found.")

        except FileNotFoundError:
            print("No student records
found.")

    def delete_student(self):
        print("\n========== DELETE
STUDENT ==========")

        delete_id = input("Enter Student ID:
")

        try:
            with open(self.FILE_NAME, "r") as
file:
                students = file.readlines()

            found = False

            with open(self.FILE_NAME, "w")
as file:
```

```
            for student in students:
                data =
student.strip().split(",")

                if data[0] == delete_id:
                    found = True
                else:
                    file.write(student)

        if found:
            print("Student deleted
successfully!")
        else:
            print("Student not found.")

    except FileNotFoundError:
        print("No student records
found.")

  def menu(self):
    while True:
```

```
        print("\n========== STUDENT
MANAGEMENT SYSTEM ==========")
        print("1. Add Student")
        print("2. View Students")
        print("3. Search Student")
        print("4. Update Student")
        print("5. Delete Student")
        print("6. Exit")

        choice = input("Enter your choice:
")

        if choice == "1":
            self.add_student()

        elif choice == "2":
            self.view_students()

        elif choice == "3":
            self.search_student()

        elif choice == "4":
            self.update_student()
```

```
        elif choice == "5":
            self.delete_student()

        elif choice == "6":
            print("\nThank you!")
            break

        else:
            print("Invalid choice!")


# Main Program

system = StudentManagementSystem()

username, password =
system.create_login()

if system.login(username, password):
    system.menu()
```

conclusion

The Student Management System is a Python-based application developed to simplify and automate the management of student records. The system providesfeatures such as secure login, adding, viewing, searching, updating, deleting, and counting student records through a simple menu-driven interface.By using Object-Oriented Programming (OOP) concepts and file handling, the application stores student information permanently and ensures easy access to records whenever required. The system reduces manual work, improves accuracy, saves time, and provides an organized way of maintaining student data.Overall, the Student Management System is an efficient, user-friendly, and reliable solution for educational institutions to manage student information effectively.